Tema 4
Verificación y validación

Pruebas de software

Contenidos

1. Motivación.
2. Testing.
3. Pruebas de caja negra.
4. Pruebas de caja blanca.
5. Buenas prácticas y herramientas.

1. Motivación

¿Qué es la verificación y validación de software?

1. Motivación – Calidad en el software

El software que escribimos contiene errores.

🎯 **Objetivo de la calidad en el Software:**
Entregar software sin defectos.

- Para conseguir calidad, no solo necesitamos testing.
- Pero el testing es fundamental.

> Quality is not equal to test. Quality is achieved by putting development and testing into a blender and mixing them until one is indistinguishable from the other.
>
> Testing must be an unavoidable aspect of development, and the marriage of development and testing is where quality is achieved.

"How Google Tests Software", James Whittaker Jason Arbon y Jeff Carollo, Addison-Wesley, 2012

1. Motivación – Terminología

Error

(humano)

- La causa de un defecto.
- Un error de diseño, un error de implementación, una errata, etc.

→ **Defecto**

(código)

- Se refiere a la parte del código que es incorrecta.
- Condición necesaria, pero no suficiente para que haya un fallo.

```
if (false)
  x = 1/0
```

→ **Fallo**

(comportamiento observable)

- Comportamiento incorrecto del software.
- Se manifiesta cuando el sistema está en funcionamiento.

1. Motivación – Ejercicio

```
int doblarValor(int i) {
  int resultado = i * i;
  return resultado;
}
```

La llamada a `doblarValor(3)` devuelve `9`. Esto es:

- ☐ Un fallo.
- ☐ Un defecto.
- ☐ Un error.

1. Motivación – Ejercicio

```
int doblarValor(int i) {
  int resultado = i * i;
  return resultado;
}
```

La llamada a `doblarValor(3)` devuelve 9. Esto es:

- [x] Un fallo.
- [ ] Un defecto.
- [ ] Un error.

Es un fallo porque es un comportamiento incorrecto del programa.
Al hacer la llamada todavía no sabemos dónde está el defecto.

DUDAS:
¿En qué línea está el defecto? ¿En qué línea está el error?

1. Motivación – Verificación y validación

Verificación
¿Se está construyendo el producto correctamente?

Validación
¿Se está construyendo el producto correcto?

Diferencia es sutil pero importante:

- Verificación: orientada al desarrollo del software y al uso de técnicas de desarrollo.
  - Código, diseño, testing, métricas de calidad, revisiones, …
  - Orientado a la calidad interna.
- Validación: orientada a determinar si el software cumple los requisitos del cliente.
  - Requisitos, casos de uso, aceptación, feedback del cliente, …
  - Orientado a la calidad externa.

En esta asignatura nos centramos en técnicas de verificación, aunque parte de la validación se alcanza indirectamente.

1. Motivación – Técnicas de verificación

Existen diferentes técnicas de verificación:

1. Testing.
2. Análisis estático.
3. Revisiones.
4. Pruebas formales.

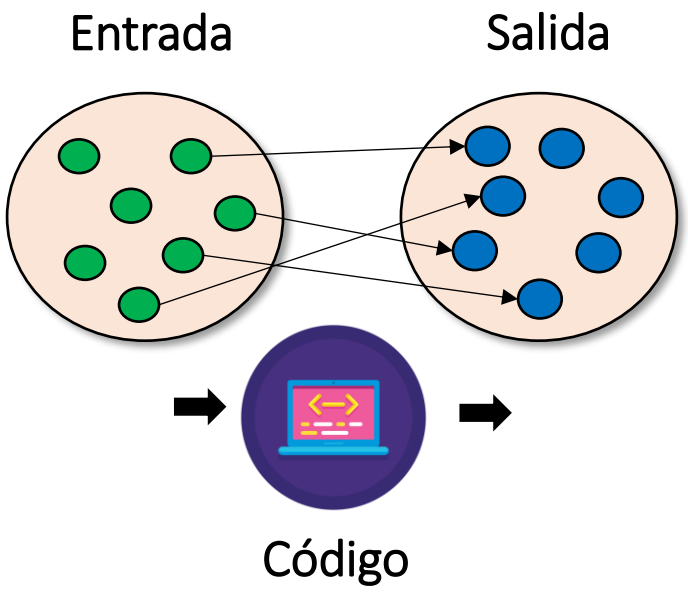
1. Motivación – Técnicas de verificación

1. Testing

- Verificación dinámica del sistema.
- El sistema se ejecutan tratando de descubrir fallos.

Ventajas e inconvenientes

- PRO: No hay falsos positivos
  si hay un fallo, es que es un fallo.
- CON: Incompleto.
  no se prueban todas las posibles ejecuciones.



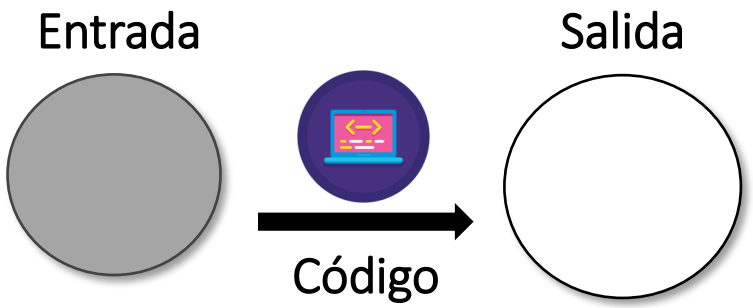
1. Motivación – Técnicas de verificación

2. Análisis estático

- Detecta problemas sin ejecutar software.
- Considera todas las posibles ejecuciones.

Ventajas e inconvenientes

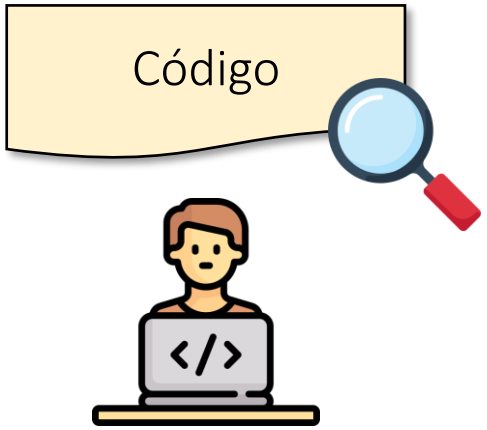
- PRO: **Considera todas** las posibles ejecuciones.
- CON: Puede generar **falsos positivos**.



1. Motivación – Técnicas de verificación

3. Revisiones

- Revisión de código y documentos.
- Revisión por otro desarrollador.



Ventajas e inconvenientes

- PRO: Sistemáticas.
  Pueden ser exhaustivas.
- CON: Informales y subjetivas.

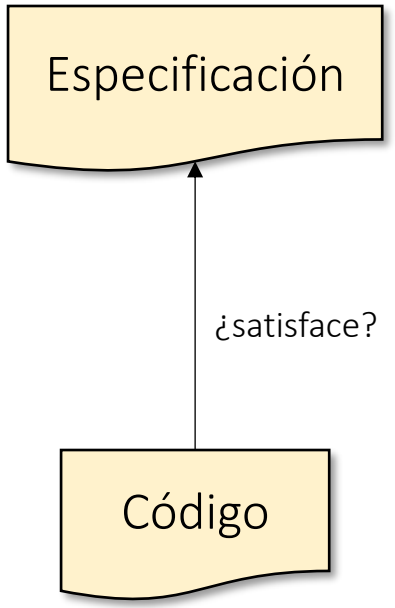
1. Motivación – Técnicas de verificación

4. Pruebas formales

- Demostración matemática.
- Cumplimiento de especificaciones.

Ventajas e inconvenientes

- PRO: Garantías fuertes de corrección.
- CON: Complejas y costosas.
  requieren escribir especificaciones.



2. Testing

1. Motivación – ¿Qué es el testing?

Testing es el proceso de ejecutar un sistema con la intención de encontrar fallos.

Proceso

- Ejecutamos el software.
- Observamos su comportamiento.
- Lo comparamos con lo esperado.

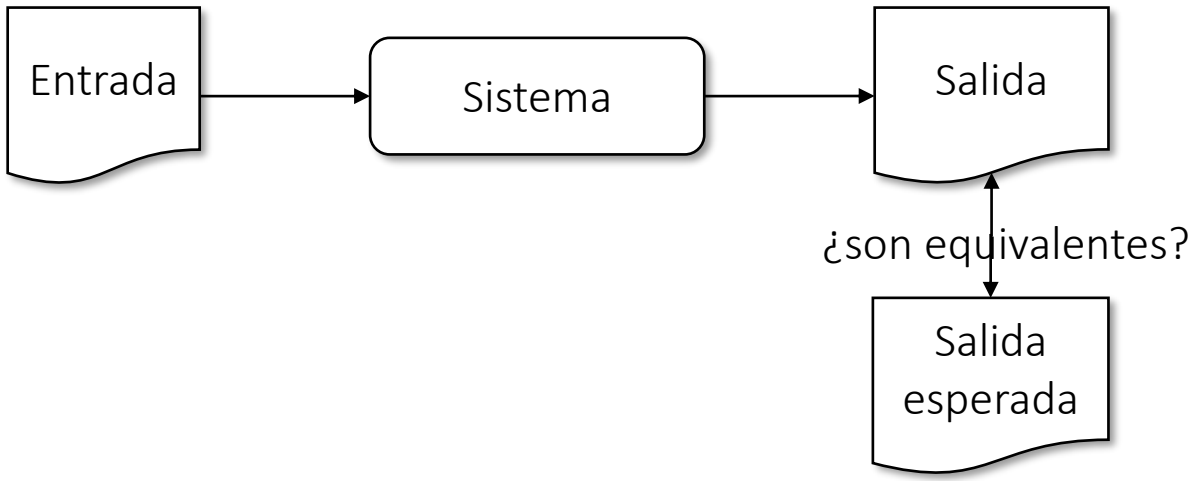
Un **test exitoso** es aquel que hace fallar el programa.

2. Testing – Motivación

En el testing estimulamos el sistema que queremos probar (System under test; SUT) mediante la ejecución de casos de prueba.

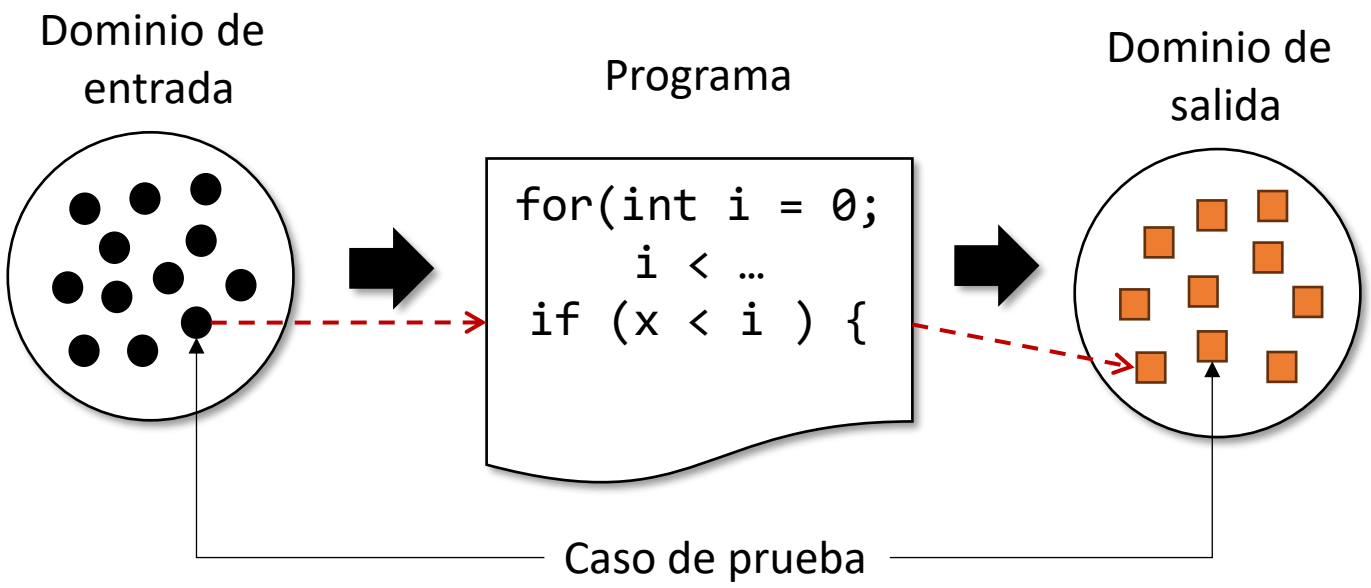
Elementos necesarios:

- **Datos de entrada**
  Datos para estimular el sistema para que produzca una salida.
- **Oráculo**
  La salida esperada o una manera de comprobar que la salida obtenida es válida.



2. Testing – Motivación

- El testing es **dinámico.**
  Es necesario ejecutar el programa para poder
- El testing es **optimista**.
  Si falla para un cierto dato, fallará para datos similares.
- El testing ejercita una (pequeña) fracción del dominio de entrada.
- Caso de prueba: `par(entrada, salida)`



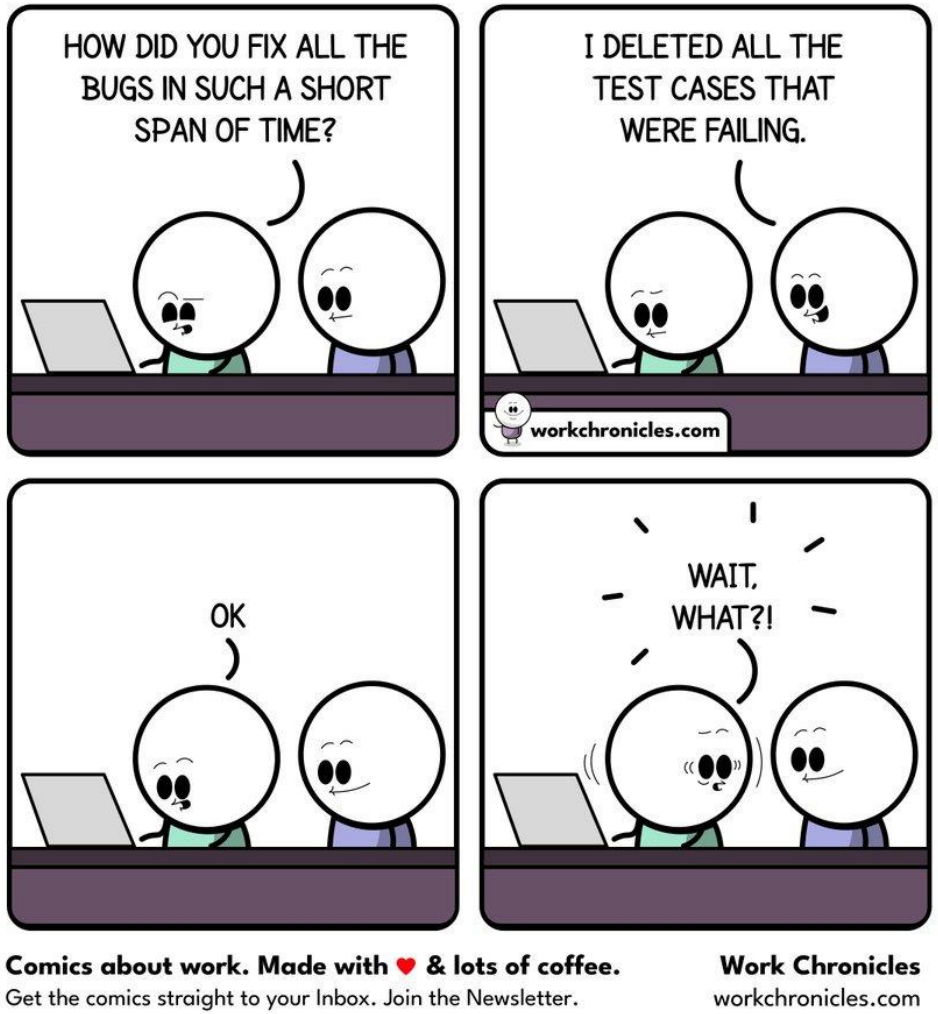
El caso de prueba detecta un error.

2. Testing – Objetivos

Un test exitoso es el que consigue hacer fallar al programa.

- El objetivo del testing no es demostrar que el software funciona.
- Es encontrar comportamientos incorrectos.
- Si un test no detecta fallos, no sabemos si el sistema es robusto.
- Un test que revela un fallo aporta información valiosa.

> “Las técnicas de testing pueden probar la existencia de errores, pero no su ausencia”
>
> Edsger Dijkstra



2. Testing – Principios

- **Presencia ≠ Ausencia.**
  Que todos los tests pasen no implica que no haya defectos.
- **Exhaustividad imposible.**
  Un formulario con 10 campos si/no tiene 2^{10} combinaciones.
- **Detectar pronto es más barato.**
  Un defecto en desarrollo más fácil de arreglar que un defecto en producción.
- **Defect Clustering.**
  La mayoría de los defectos se concentran en pocos módulos.
- **Paradoja del pesticida.**
  Si repites los mismos tests, dejan de encontrar fallos nuevos.
- **Depende del contexto**
  No se prueba igual un videojuego que un banco.
- **Falacia de la ausencia de errores**
  Ausencia de errores no implica utilidad
  Validación ≠ verificación.

* International Software Testing Qualifications Board (ISTQB)

2. Testing – Niveles

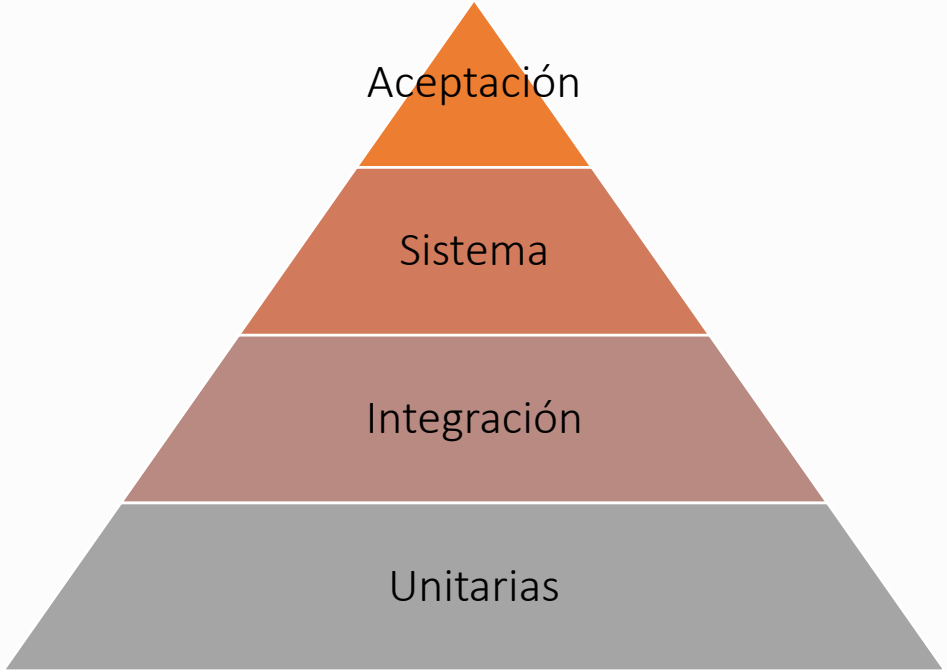
Pruebas Unitarias
Comprueban la funcionalidad de un módulo.

Pruebas de Integración
Comprueban los módulos al integrarse.

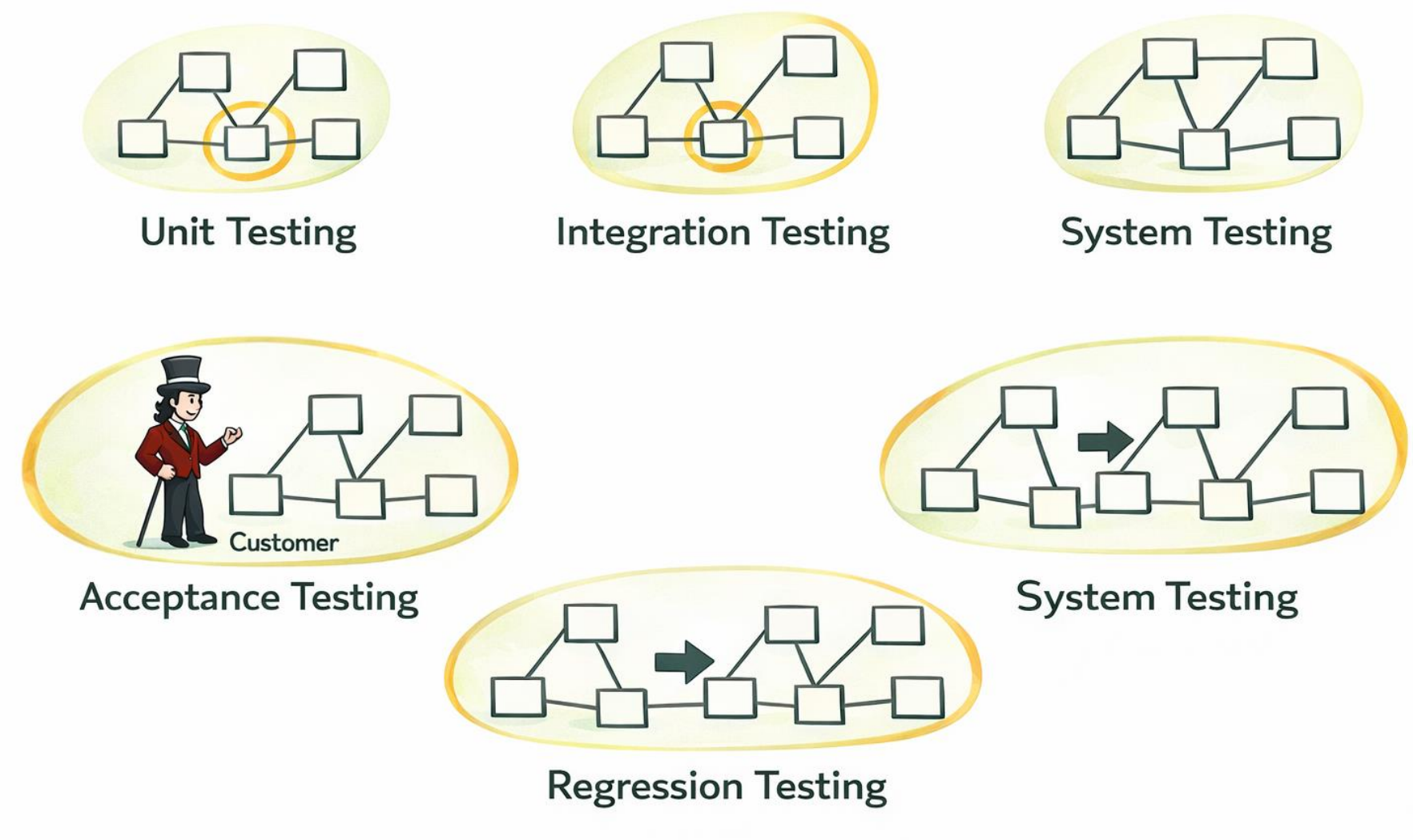
Pruebas de Sistema
comprueban requisitos no funcionales.

Pruebas de Aceptación
Comprueban requisitos del cliente.

Pruebas de regresión
Ejecutan todos los tests cuando se modifica el sistema.



2. Testing – Niveles



2. Testing – Niveles – Pruebas unitarias

Objetivo:

Probar unidades de código de manera aislada.

- Normalmente un método o una clase.
- Probar una única característica del código.
- Existen distintos frameworks (junit, phpunit, …) para casi todos los lenguajes.

Esquema básico

```
Para cada caso de prueba
  Configurar el entorno
  Ejecutar test.
  Limpiar el entorno
```

2. Testing – Niveles – Pruebas unitarias

xUnit

- Familia de frameworks para testing automatizado.
- Disponible en prácticamente todos los lenguajes.
- Popularizado por Kent Beck y Erich Gamma.
- Estructura clara para escribir tests.
  - Automatización de ejecución.
  - Asserts para comparar resultados.
  - Integración con IDE y CI/CD.

JUnit 5

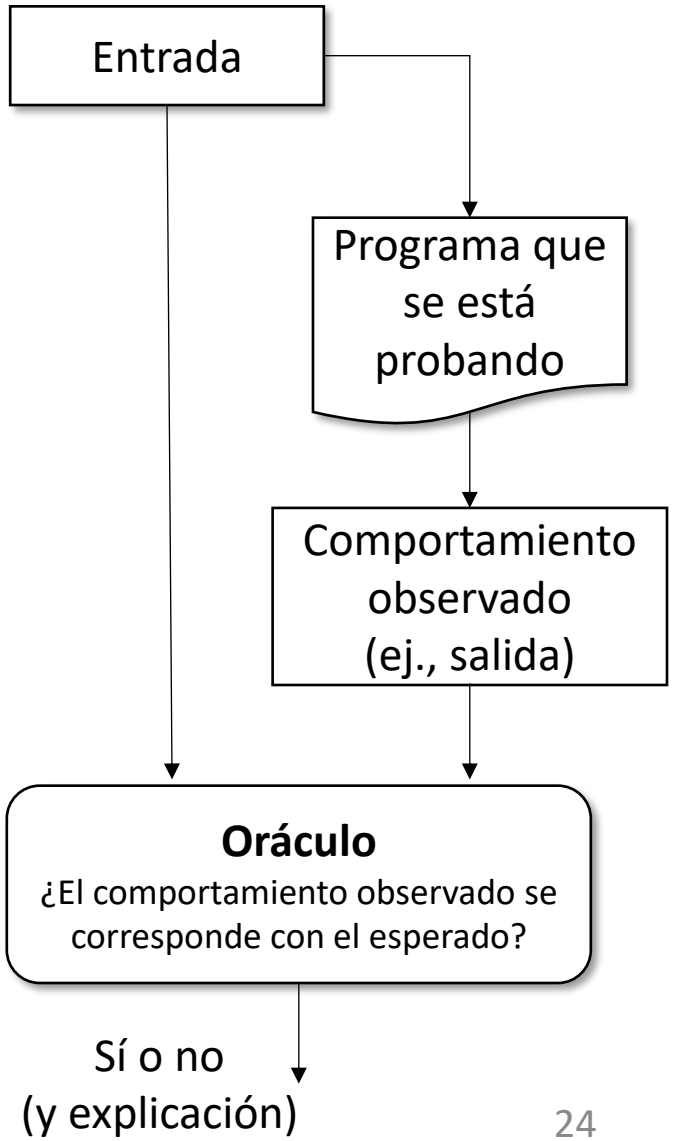
2. Testing – Niveles – Pruebas unitarias

En las pruebas de software, un **oráculo** nos ayuda a determinar si el resultado de una prueba es correcto o no.

- Puede ser tan simple como un valor de salida asociado a uno de entrada.
- Puede ser un programa diseñado para chequear el comportamiento de otros programas.

Ejemplo:

- En JUnit típicamente usamos `assert` para indicar el oráculo



2. Testing – Niveles – Pruebas de integración

Objetivo

Probar la interacción entre los módulos.

- Interés en problemas relativos a la coordinación entre módulos.
- Estas pruebas se realizan después las pruebas unitarias.

Las pruebas de integración en DDD:

- Implican probar las interacciones entre agregados, repositorios y otros servicios del dominio.
- Las pruebas de integración garantizan que sus agregados y servicios colaboren como se espera.

2. Testing – Niveles – Pruebas de integración

Estrategias de pruebas de integración

1. Todos a la vez – *"big bang integration testing"*
   - Configurar el sistema completo.
   - Se parece más a Pruebas de sistema.
   - La arquitectura hexagonal las facilita
2. Incremental.
   - Probar las interacciones poco a poco.
   - Se pueden ejercitar partes reales del sistema (*ej., una base de datos*)
   - Otras partes, se pueden configurar como *mocks.*

2. Testing – Niveles – Pruebas de integración

Ejemplo DDD

```
@Test
public void testPedidoRechazadoSiUsuarioBloqueado () {

  // Preparamos los datos y configuramos el escenario
  Usuario usuario = new Usuario("john@doe.com", EstadoUsuario.BLOQUEADO);
  Pedido pedido = new Pedido (usuario.getId(), restaurante);
  Plato pizza = new Plato (“Pizza", "pizza grande", 20.0);

  pedido.addLineaPedido (new LineaPedido (pizza, 2));

  // Arrancamos el servicio
  ServicioPedidos servicio = new ServicioPedidos();
  servicio.realizarPedido(usuario, pedido);

  // Hacemos la comprobación
  assertEquals(EstadoPedido.RECHAZADO, pedido.getEstado());
}
```

Ejemplo:
Servicio de dominio.

- Probamos la coordinación entre dos agregados.
- Supongamos un dominio con dos agregados: **`usuario`** y **`pedidos`** con la siguiente regla de negocio: “*Un usuario bloqueado no puede realizar pedidos*”.

NOTA: Se podría ver como una prueba unitaria del módulo ServicioPedidos.

2. Testing – Niveles – Pruebas de integración

```java
@Test
public void testPedidoSeRealizaCorrectamente () {

  // Configuramos el escenario
  UsuarioRepository usuarioRepo = new UsuarioRepositoryInMemory();
  PedidoRepository pedidoRepo = new PedidoRepositoryInMemory();

  Usuario usuario = new Usuario("john@doe.com", EstadoUsuario.ACTIVO);
  usuarioRepo.save(usuario);

  Pedido pedido = new Pedido(usuario.getId(), restauranteId);
  pedidoRepo.save(pedido);

  // Servicio de aplicación
  RealizarPedidoService appService = new RealizarPedidoService(usuarioRepo, pedidoRepo);

  appService.realizarPedido(usuario.getId(), pedido.getId());

  Pedido pedidoActualizado = pedidoRepo.findById(pedido.getId());

  assertEquals(EstadoPedido.CONFIRMADO, pedidoActualizado.getEstado());

}
```

Ejemplo:
Servicio de aplicación.

Se coordinan:

- Repositorios.
- Agregados.
- Reglas de negocio.

- Estamos verificando que el flujo de datos funciona.
- No dependemos de BD, red, Docker, etc.

2. Testing – Niveles – Pruebas de sistema

Pruebas *end-to-end (E2E)*

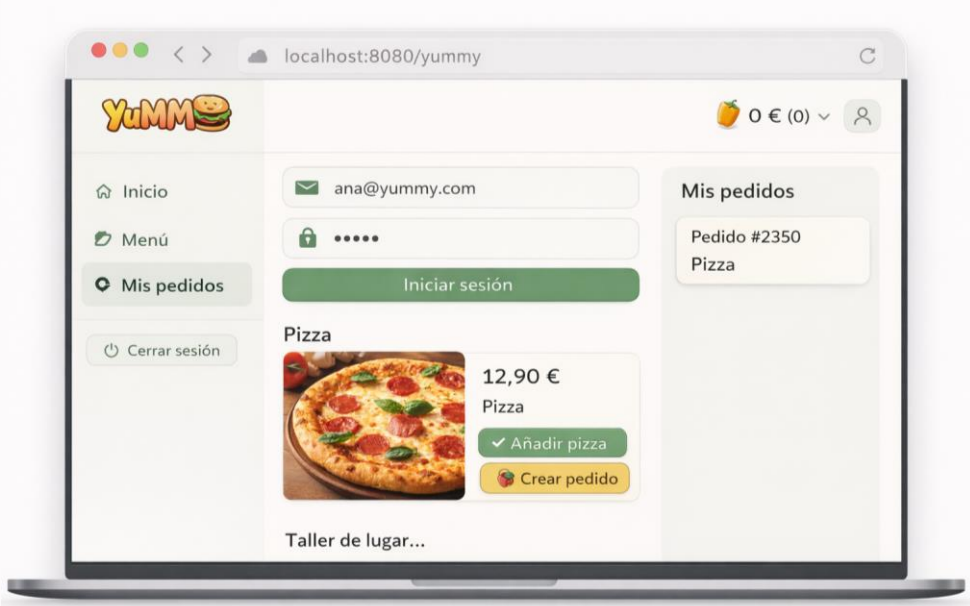
- Simulan el uso de un sistema por un usuario.
- El sistema **como un todo**: *UI* → *backend* → *BD* → *respuesta*
- Pueden ser costosas de ejecutar.
- En ocasiones, herramientas especializadas como *Selenium* o *Playwright*.

Dudas:

- ¿Cómo sería una prueba de sistema para una aplicación web?
- ¿Cómo sería una prueba de sistema para un compilador?

2. Testing – Niveles – Pruebas de sistema – Ejemplo

```
@Test
void login_y_crearPedido_apareceEnMisPedidos() {
  driver.get("http://localhost:8080/yummy");

  // Login
  driver.findElement(By.id("email")).sendKeys("ana@yummy.com");
  driver.findElement(By.id("password")).sendKeys("1234");
  driver.findElement(By.id("btn-login")).click();

  // Crear pedido
  driver.findElement(By.id("add-item-pizza")).click();
  driver.findElement(By.id("btn-confirmar-pedido")).click();

  // Oráculo: aparece en "Mis pedidos"
  driver.findElement(By.id("menu-mis-pedidos")).click();
  String texto = driver.findElement(By.cssSelector(".pedido:first-child")).getText();

  assertTrue(texto.contains("Pizza"));
}
```



2. Testing – Niveles – Pruebas de regresión

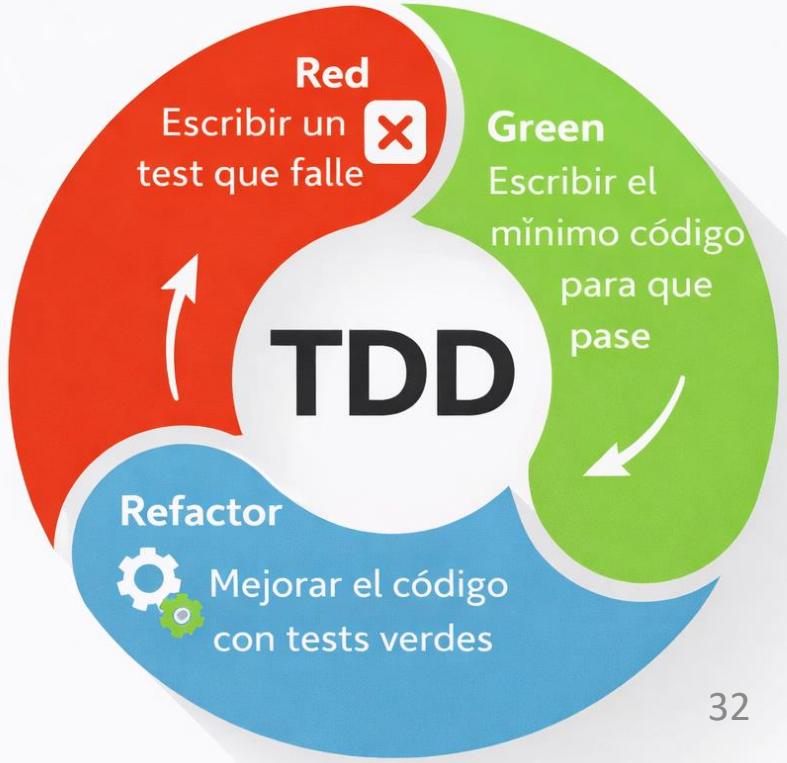
Las **pruebas de regresión** hacen referencia a la estrategia de pruebas utilizada para ejecutar las pruebas existentes cada vez que se añade nueva funcionalidad o se arregla un defecto

- Cualquier cambio en el software puede introducir problemas en otras partes
  - Especialmente si el software no está bien modularizado
- Siempre hay que ejecutar de nuevo las pruebas que tenemos
- Si el *test suite* es muy grande
  - Priorizar y seleccionar pruebas

2. Testing – Niveles – Pruebas de regresión

El TDD (*Test Driven Development*) es el desarrollo guiado por pruebas.

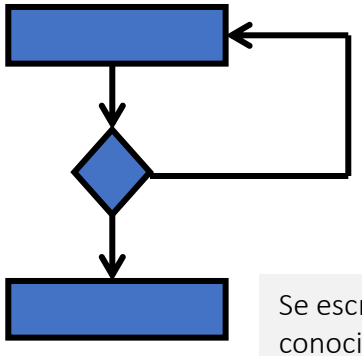
- El test se escribe antes que el código.
- Los tests especifican el comportamiento.
- Obliga a pensar en el diseño antes de implementar.
- Reduce código innecesario.
- Fomenta diseño desacoplado y testeable.
- Genera suite de regresión automáticamente.



2. Testing – Tipos de pruebas

- **Pruebas de caja negra** *(AKA: testing funcional o basado en la especificación.)*
  - Diseñadas sin conocer la implementación.
  - Basadas en especificaciones externas del software (requisitos funcionales de la aplicación, descripción de interfaces, diseño, etc.)

- **Pruebas de caja blanca** *(AKA: testing estructural.)*
  - Examina la implementación del sistema.
  - Las pruebas derivan de la estructura del código fuente a probar: bifurcaciones, iteraciones, llamadas a funciones, …



Se escriben conociendo el código, por ejemplo para asegurar que se cubren todas las líneas

3. Pruebas de caja negra

Pruebas basadas en la especificación

3. Pruebas de caja negra

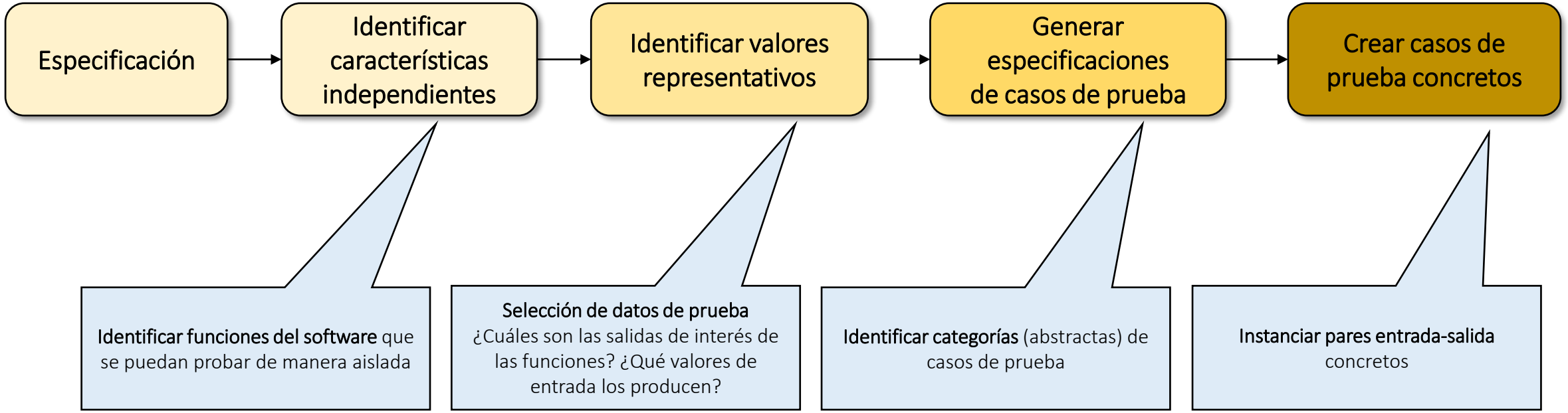
Las **pruebas de caja negra** se centran en el comportamiento del sistema, sin tener en cuenta su implementación interna.

Características

- Se toma como base la especificación.
- Centradas en el dominio del problema, no en la implementación.
  - Capturan errores lógicos.
  - Se centra en los *comportamientos importantes* del software (los que están especificados)
- No es necesario conocer el código o que esté implementado.
- Se pueden aplicar a cualquier nivel de pruebas.

3. Pruebas de caja negra – Testing funcional

Partimos de una especificación del comportamiento esperado del software (ej., *los requisitos*)



3. Pruebas de caja negra – Identificación características

La **identificación de características independientes** depende del nivel al que estemos probando:

- **Pruebas unitarias.**
  Las características se descompondrán a nivel de métodos o clases.
- **Pruebas de integración.**
  Las características de descompondrán a nivel de módulos.
- **Pruebas de sistema.**
  Interfaces de usuario o APIs.

3. Pruebas de caja negra – Selección de datos de prueba

¿Cómo seleccionamos nuestros datos de prueba?

```
public boolean aplicarDescuento(long precio)
```

Podemos usar un enfoque *naive,* probando todos los posibles valores de **precio**.

Pero… **precio** es **long**.
Tiene 2^{64} posibles valores distintos (1.8×10^{19} apróx.)

Si hiciéramos una prueba por nanosegundo, tardaríamos aproximadamente 570 años.

"Para cualquier sistema de tamaño real, el número de casos de prueba posibles es prácticamente infinito. No tenemos tiempo suficiente para ejecutarlos todos.
Es necesario seleccionar un subconjunto de casos de prueba."

* Se usa long solo para simplificar (con un double la situación es mucho más complicada)

3. Pruebas de caja negra – Selección de datos de prueba

Random testing

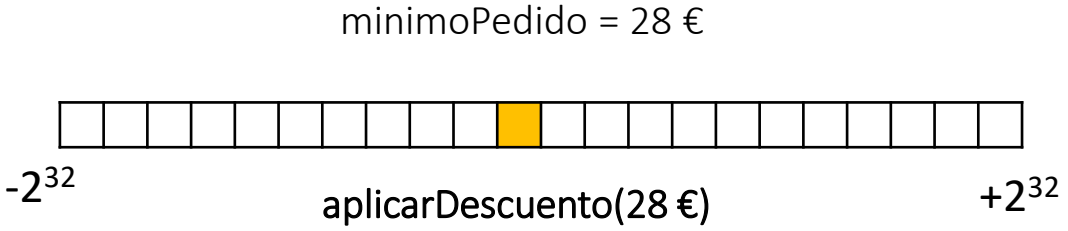
- Consiste en elegir entradas aleatoriamente
  - Cualquier entrada es "*igual de buena*" que otra para encontrar un error.
  - No hay sesgo del tester ni de malinterpretar la especificación.

Pero… **Los defectos no se distribuyen de manera uniforme.**

Especificación de ***`aplicarDescuento(precio)`***: "Se aplicará el descuento si el precio es mayor o igual que el pedido mínimo especificado en la configuración del sistema"

Si escribiste: `(precio > minimoPedido)` y no `(precio >= minimoPedido)`

¿Cuál es la posibilidad de encontrar ese bug eligiendo valores al azar?



3. Pruebas de caja negra – Partition testing

- Elegir “manualmente” entradas que sean especialmente relevantes.
- En la práctica, los errores similares se acumulan en la misma zona.

El partition testing se basa en la idea de que algunos conjuntos de entradas hacen que el programa se comporte igual independientemente de los valores concretos.

3. Pruebas de caja negra – Partition testing

Un conjunto de valores de entrada son una clase de equivalencia **(equivalence partition)** si se puede asumir de manera razonable que provocarán salidas "*similares*" al ejecutar la implementación.

Objetivo

- Definir grupos de entradas que provocan el mismo comportamiento en el programa.
  1. Leer la especificación
  2. Definir particiones
- Generación de casos de prueba concretos
  Elegir un valor de cada partición. Todos son equivalentes.

```
int abs(int valor) { … }
```

Clases de equivalencia

1. **valor < 0:** abs(-37) = 37, abs(-36) = 36, etc.
2. **valor > 0:** abs(10) = 10, abs(11) = 11, etc.
3. **valor = 0:** No es ninguna de las anteriores. abs(0) = 0

3. Pruebas de caja negra – Clases válidas e inválidas

Las entradas se pueden categorizar en:

- Clase válida: Entradas que el sistema debería aceptar y procesar correctamente.
- Clase inválida: Entradas que no cumplen los requisitos y el sistema debería rechazar con un error

Si la especificación incluye condiciones sobre la entrada *(ej., el valor tiene que ser mayor que 0)*, usar esta información para derivar las clases válidas e inválidas.

- Clase válida: dentro del rango.
- Clase inválida: fuera del rango (muy pequeño).
- Clase válida: fuera del rango (demasiado grande).

3. Pruebas de caja negra – Clases válidas e inválidas

- Las entradas inducen clases de equivalencia sobre entradas válidas e inválidas

Tipo de entrada	Clases de equivalencia
Rango	1 válida, 2 inválidas (mayor y menor)
Valor	1 válida, 2 inválidas (mayor y menor)
Miembro de un conjunto	1 válida, 1 inválida
Booleano	1 válida, 1 inválida

- Edad permitida para registrarse: 18 a 65 años
  - Clase válida: 18–65. Clase inválida (menor): < 18. Clase inválida (mayor): > 65
- Campo “Acepta términos y condiciones”
  - Clase válida: true. Clase inválida: false

3. Pruebas de caja negra – Partition testing

- **Partition testing** nos ayuda a encontrar rangos de valores que son interesantes
  - Pero no podemos elegir todos los valores de la partición
- Necesitamos elegir al menos un valor concreto de cada partición
- ¿Qué valores usar en realidad?

3. Pruebas de caja negra – Boundary value analysis

- Los errores tienden a ocurrir en los límites del dominio (*o sub-dominio*).
- Podemos seleccionar valores concretos que exploren esos límites

Especificación: los menores de 18 tienen descuento para entrar a un parque.

```
public boolean tieneDescuento(int edad)
```

Regla:

- Si edad < 18 → no tiene descuento.
- Si edad >= 18 → no tiene descuento.

En este punto el programa cambia su comportamiento.
Hay dos particiones.

- **Límite:** 18
- **Clase menor:** 17
- **Clase mayor:** 19

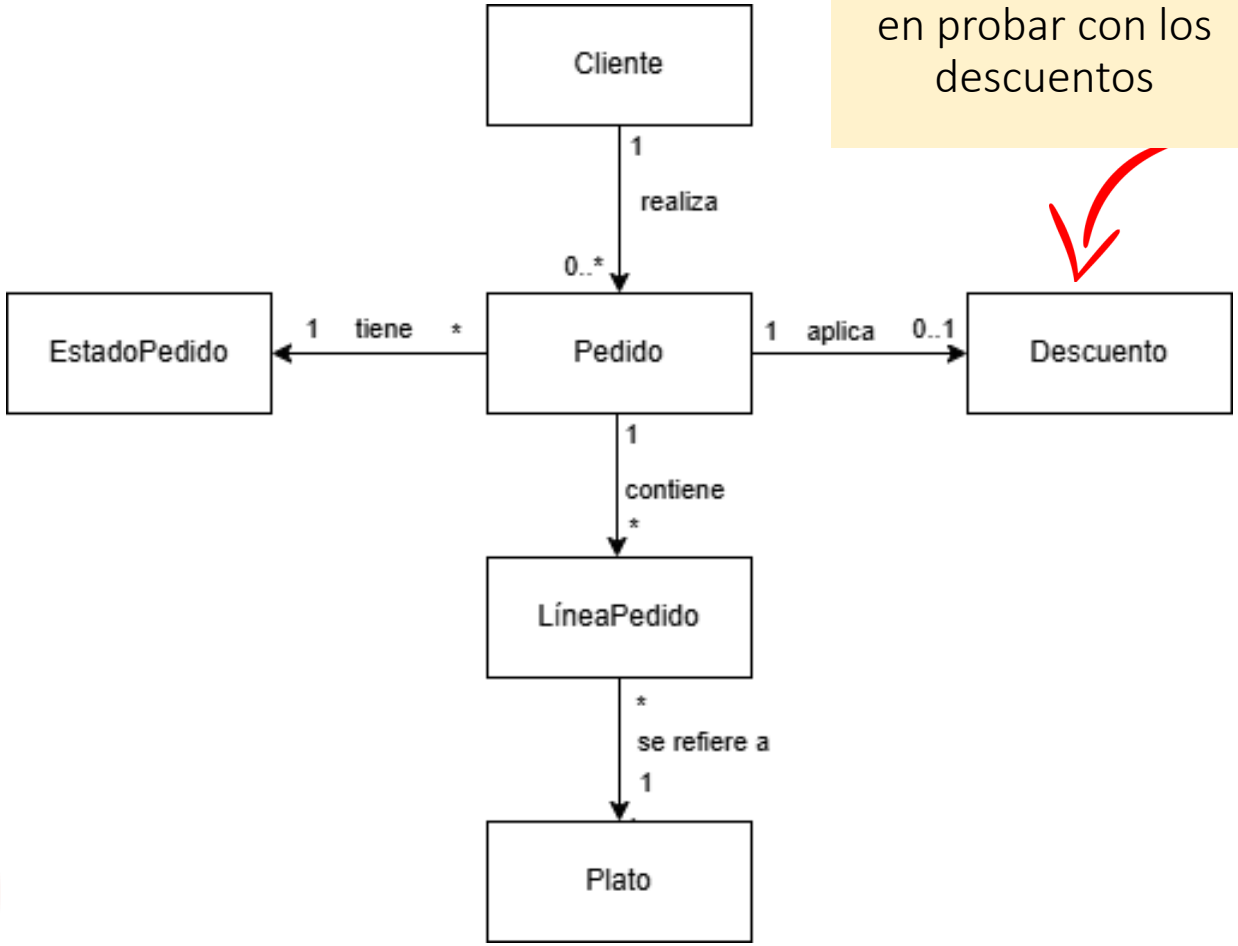
3. Pruebas de caja negra – Ejercicio

Vamos a ver esto con un caso de estudio.

La UMU quiere implantar un servicio de comida rápida y a domicilio, llamado **yUMmy**, pensado especialmente para estudiantes que pasan muchas horas en la universidad y no siempre tienen tiempo para cocinar o salir a comer.

La idea es ofrecer una aplicación sencilla y eficiente para que los estudiantes puedan hacer pedidos de comida a los restaurantes del campus o cercanos, de forma rápida, con precios especiales y descuentos para la comunidad universitaria.

Vamos a centrarnos en probar con los descuentos



3. Pruebas de caja negra – Ejercicio

¿Qué vamos a probar?:

La clase ***PoliticaDescuento*** *aplica un porcentaje de descuento a un pedido si este supera el importe mínimo. Si no se cumple la condición, devuelve el precio original.*

```
public class PoliticaDescuento {
  private double minimoPedido;
  private double porcentaje;

  public PoliticaDescuento(double minimo, double porcentaje) {
    this.minimoPedido = minimo;
    this.porcentaje = porcentaje;
  }

  /**
   * Se aplica un descuento si el precio del pedido es
   * es mayor o igual que el importe mínimo (minimoPedido).
   */
  public double aplicarDescuento (double precioBase) {
    // La implementación no es relevante
  }
}
```

3. Pruebas de caja negra – Ejercicio

testDescuentoAplicado

Verificar que cuando el precio del pedido es mayor que 20 €, el descuento se aplica correctamente.

- Descuento disponible
  10% en pedidos a partir de 20 €.
- Precio base del pedido
  25 €.
- Descuento esperado
  10% de 25 € = 2.5 €.
- Precio final esperado
  22.5 €.

```
@Test
public void testDescuentoAplicado () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 0.10);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (25.0);

  // (3) Valido mi prueba
  assertEquals (22.5, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testSinDescuentoAplicado

Verificar que cuando el pedido es menor que 20 €, NO se aplica ningún descuento.

- Descuento disponible
  10% en pedidos a partir de 20 €.
- Precio base del pedido
  15 €.
- Descuento esperado
  (no aplica).
- Precio final esperado
  15 €.

```
@Test
public void testSinDescuentoPorBajoMinimo () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 0.10);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (15.0);

  // (3) Valido mi prueba
  assertEquals (15.0, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testPorcentajeCero

Verificar que cuando el porcentaje de descuento es 0, no se aplica descuento.

- Descuento disponible
  0% en pedidos a partir de 20 €.
- Precio base del pedido
  30 €.
- Descuento esperado
  0 €.
- Precio final esperado
  30 €.

```
@Test
public void testPorcentajeCero () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 0.0);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (30.0);

  // (3) Valido mi prueba
  assertEquals (30.0, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testDescuentoTotal

Verificar que cuando el porcentaje de descuento es 100%, el pedido queda a 0 €, siempre que supere el mínimo.

- Descuento disponible
  100% en pedidos a partir de 20 €.
- Precio base del pedido
  50 €.
- Descuento esperado
  0 €.
- Precio final esperado
  30 €.

```
@Test
public void testDescuentoTotal () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 1.0);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (50.0);

  // (3) Valido mi prueba
  assertEquals (0.0, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testSinDescuentoMinimo

Verificar que puedo tener descuentos sin precio mínimo.

- Descuento disponible
  10% en todos los pedidos.
- Precio base del pedido
  5 €.
- Descuento esperado
  10% de 5 € = 0,5 €.
- Precio final esperado
  4,5 €.

```
@Test
public void testSinDescuentoMinimo () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (0.0, 0.1);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (5.0);

  // (3) Valido mi prueba
  assertEquals (4.5, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testPorcentajeFueraDeRango

Verificar que no existan porcentajes por encima y por debajo del mínimo .

```
@Test
public void testPorcentajeFueraDeRango () {
  assertThrows (IllegalArgumentException.class, () -> new Descuento (20.0, 1.5));
  assertThrows (IllegalArgumentException.class, () -> new Descuento (20.0, -0.1));
}
```

3. Pruebas de caja negra – Ejercicio

NUEVA REGLA DE NEGOCIO

A partir de ahora, el descuento que se aplique no puede superar los 100 € por pedido, aunque el porcentaje aplicado sea muy alto.

¿Son suficientes los tests que teníamos para las nuevas reglas de negocio?

```
public class Descuento {
  …;
  private static final double MAXIMO_DESCUENTO = 100.0;

  public double aplicarDescuento (double precioBase) {
    if (precioBase >= minimoPedido) {
      double descuento = precioBase * porcentaje;
      if (descuento > MAXIMO_DESCUENTO) {
        descuento = MAXIMO_DESCUENTO;
      }
      return precioBase - descuento;
    } else {
      return precioBase;
    }
  }
}
```

3. Pruebas de caja negra – Ejercicio

Reflexiones y estrategias

- Tiene sentido comprobar un valor representativo y los valores límite.
- Los valores representativos nos ayudan a ver como se comporta el sistema con valores típicos.
- Siempre interesa al menos tener una prueba de frontera para cada valor.
- También añadir pruebas con valores límite.

4. Pruebas de caja blanca

Testing estructural

4. Pruebas de caja blanca – Definición

Las **pruebas de caja blanca** (o ***testing estructural***) son aquellas que se derivan del conocimiento el código fuente.

Motivación:

- Para encontrar un defecto, el código que lo contiene debe ejecutarse.
- Probar **funcionalidad no basta**.
  Debemos recorrer la estructura interna del código.
- Las pruebas de caja blanca nos ayudan a diseñar casos que ejerciten caminos y decisiones del código.

4. Pruebas de caja blanca – Motivación

Si partimos solo de la especificación…

¿Por qué puede quedar código sin probar?

Posibles razones.

- El desarrollador cometió un error.
  - L*a especificación era clara, pero la implementación no la respeta*.
- La especificación era incompleta o ambigua.
  - El desarrollador añade comportamiento no previsto explícitamente.
  - El tester no sabía que debía probarlo.
- Existen decisiones de implementación no visibles en la especificación.
  - Validaciones internas, manejo de errores, optimización.
  - Estas partes del código pueden no estar cubiertas por pruebas basadas solo en requisitos.

4. Pruebas de caja blanca – Cobertura

Las pruebas de caja blanca se apoyan en **criterios y métricas de cobertura**, que miden **qué proporción del código ha sido ejecutada** por nuestras pruebas.

- Las pruebas de caja blanca utilizan criterios de cobertura estructural para evaluar el porcentaje de Sentencias, Decisiones, Condiciones o Caminos del programa.
- Estas métricas no garantizan ausencia de defectos, **pero permiten detectar partes del código que nunca han sido ejercitadas**.

4. Pruebas de caja blanca – Ejemplo

EJEMPLO: Contar palabras de una frase que terminan en ‘s’ o ‘r’

```
public int count(String sentence) {
    int words = 0;
    char last = ' ';
    for (int i = 0; i < sentence.length(); i++) {
      if (!isLetter(sentence.charAt(i)) &&
          (last == 's' || last == 'r')) {
        words++;
      }
      last = sentence.charAt(i);
    }
    if (last == 'r' || last == 's') {
      words++;
    }
}
```

Si el carácter actual no es una letra y el anterior era ‘s’ o ‘r’, la palabra anterior acababa en ‘s’ o ‘r’

¿Qué líneas ejecuta cada test?

```
@Test
void twoWordsEndingWithS() {
  int words = count("dogs cats");
  assertThat(words).isEqualTo(2);
}

@Test
void noWordsAtAll() {
  int words = count("dog cat");
  assertThat(words).isEqualTo(0);
}
```

4. Pruebas de caja blanca – Ejemplo

Hay herramientas que miden la cobertura.

- Ejemplo: JaCoCo - https://www.jacoco.org/jacoco/

Un diamante indica que hay una instrucción de salto y puede haber varias ramas que cubrir.

El test suite no ejercita la condición last == ‘r’

```
public class CountWords {
    public int count(String str) {
        int words = 0;
        char last = ' ';
        for (int i = 0; i < str.length(); i++) {
            if (!Character.isLetter(str.charAt(i)) && (last == 's' || last == 'r')) {
                words++;
            }
            last = str.charAt(i);
        }
        if (last == 'r' || last == 's')
            words++;
        return words;
    }
}
```

El color indica si la línea se ha cubierto *(total, parcialmente o no cubierta)*

4. Pruebas de caja blanca – Ejemplo

Ahora, introducimos otra prueba que ejercite probar las líneas pendientes.

```java
@Test
void wordsThatEndInR() {
  int words = new CountWords().count("car bar");
  assertThat(words).isEqualTo(2);
}
```

All lines are green, which means all lines and branches of the method are covered by at least one test case.

```java
public class CountWords {
    public int count(String str) {
        int words = 0;
        char last = ' ';
        for (int i = 0; i < str.length(); i++) {
            if (!Character.isLetter(str.charAt(i)) && (last == 's' || last == 'r')) {
                words++;
            }
            last = str.charAt(i);
        }
        if (last == 'r' || last == 's')
            words++;
        return words;
    }
}
```

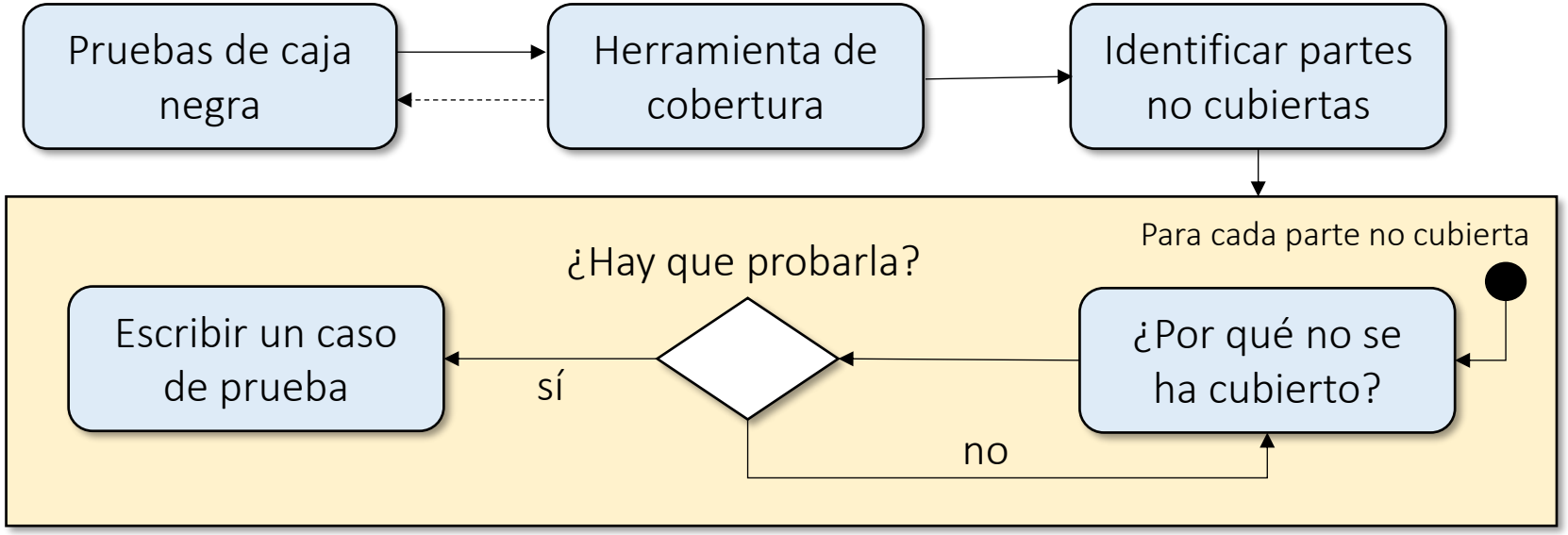
Todas las líneas en verde.
Las sentencias y ramas del método están completamente cubiertas por los casos de prueba.

4. Pruebas de caja blanca – Ventajas

Las pruebas de caja blanca tienen las siguientes ventajas:

- Pueden medirse de manera automática y objetiva.
  - Al estar basadas en el código, existen métricas para medir cómo de buenas son las pruebas con respecto al código que están probando.
- Las métricas se pueden usar para comparar test suites.
- No hay subjetividad, lo que se está probando es lo que hay en el código.
  En las pruebas de caja negra, a veces, la especificación se puede interpretar de diversas formas
- Permiten asegurar que se cubre el código que se ha desarrollado.

4. Pruebas de caja blanca – Proceso de pruebas



4. Pruebas de caja blanca – Métricas

Existen distintas métricas para medir la cobertura de nuestro código.

- 📏 Sentencias (o Líneas)
- 🔦 Condiciones
- 🌿 Ramas
- 🌿 + 🔦 Ramas y condiciones
- 🎯 MC/DC
- 🧠 Múltiples condiciones

4. Pruebas de caja blanca – Métricas – Sentencias

$$cobertura\ de\ sentencias = \frac{sentencias\ ejecutadas}{número\ total\ de\ sentencias} \cdot 100$$

- Relación entre sentencias ejecutadas entre el total de líneas de código.
- Se cumple cuando **todas las sentencias del programa se han ejecutado**.
- Independiente de la complejidad de la sentencia.
  Cuenta lo mismo un if con muchas condiciones que una asignación.

4. Pruebas de caja blanca – Métricas – Sentencias

Aclaración: Diferencias entre línea y sentencias.

- Una línea de código no siempre coincide con una sentencia.
- Una sentencia es una instrucción ejecutable.
- Una línea puede contener varias sentencias.
- Una sentencia puede ocupar varias líneas.

Línea ≈ Sentencia *(statement)*.

```
if (condition) print ("Hola mundo");

if (condition) {
  print ("Hola mundo");
}
```

Asumimos una organización razonable de las sentencias en líneas. En ambos casos cobertura = 50%.

Normalmente las herramientas trabajan a nivel de instrucciones/sentencias.

4. Pruebas de caja blanca – Métricas – Sentencias

Ejemplo

```
imprimirSuma(int a, int b) {
  int resultado = a + b;
  if (resultado > 0) {
    impcolor("verde", resultado);
  } else if (resultado < 0) {
    impcolor("rojo", resultado);
  }
}
```

Caso de Prueba #1
a = 3
b = 9

Caso de Prueba #2
a = -5
b = -8

$$cobertura(caso\ prueba\ 1) = \frac{3}{5} \cdot 100 = 60\%$$

Ventajas

- La cobertura por sentencias es muy usada en la práctica.
- Fácil de interpretar.
- Un objetivo típico puede ser alcanzar entre un 80 – 90% de cobertura.

4. Pruebas de caja blanca – Métricas – Ramas

$$cobertura\ de\ ramas = \frac{ramas\ ejecutadas}{número\ total\ de\ ramas} \cdot 100$$

- Mide la relación entre condiciones booleanas ejecutadas entre el total de condiciones del código.
- El objetivo de la cobertura de ramas (*Branch coverage*) es ejecutar todas las ramas del programa.

¿Cómo visualizar las ramas?

- Grafo de flujo de control (*control flow graph*)
- Permite visualizar ramas "ocultas" como los `else`.

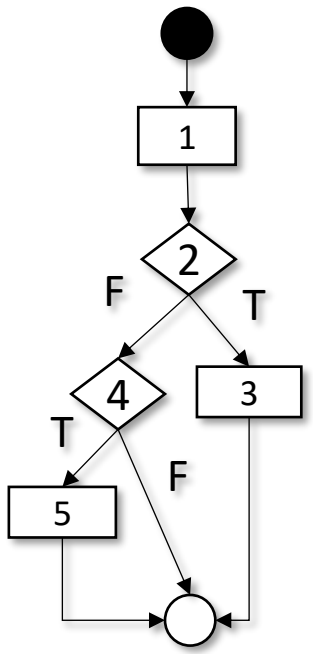
4. Pruebas de caja blanca – Métricas – Ramas

Ejemplo

```
    imprimirSuma(int a, int b) {
1     int resultado = a + b;
2     if (resultado > 0) {
3       impcolor("verde", resultado);
4     } else if (resultado < 0) {
5       impcolor("rojo", resultado);
      }
    }
```

Caso de Prueba #1
a = 3
b = 9

Caso de Prueba #2
a = -5
b = -8



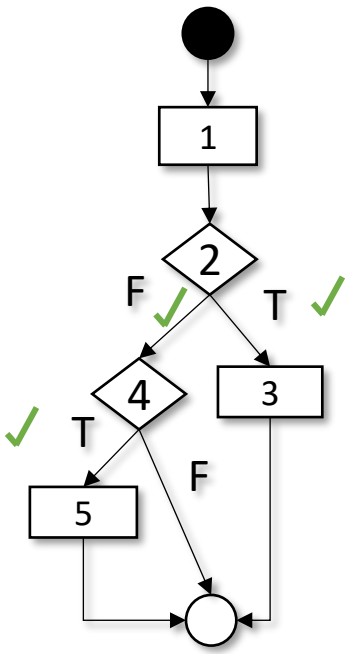
4. Pruebas de caja blanca – Métricas – Ramas

Ejemplo

```
imprimirSuma(int a, int b) {
1   int resultado = a + b;
2   if (resultado > 0) {
3     impcolor("verde", resultado);
4   } else if (resultado < 0) {
5     impcolor("rojo", resultado);
    }
}
```

Caso de Prueba #1
a = 3
b = 9

Caso de Prueba #2
a = -5
b = -8



$$cobertura(caso\ prueba\ 1{,}2) = \frac{3}{4} \cdot 100 = 75\%$$

4. Pruebas de caja blanca – Métricas – Ramas

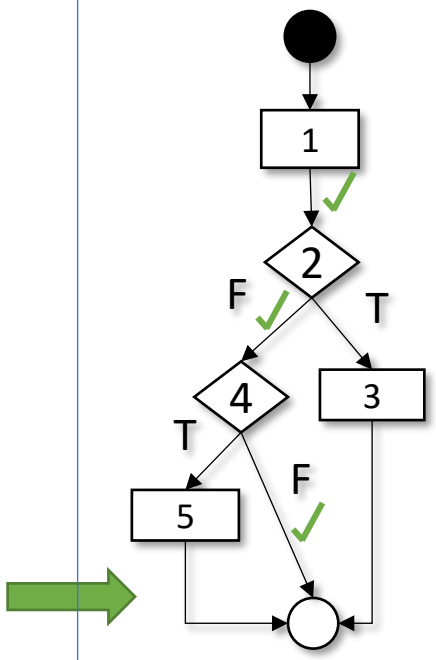
Ejemplo

```
imprimirSuma(int a, int b) {
1   int resultado = a + b;
2   if (resultado > 0) {
3     impcolor("verde", resultado);
4   } else if (resultado < 0) {
5     impcolor("rojo", resultado);
    }
}
```

Caso de Prueba #1
a = 3
b = 9

Caso de Prueba #2
a = -5
b = -8

Caso de Prueba #3
a = 0
b = 0



$$cobertura(caso\ prueba\ 1{,}2{,}3) = \frac{4}{4} \cdot 100 = 100\%$$

4. Pruebas de caja blanca – Métricas – Ramas

Subsumpción

Un criterio A subsume a B si:

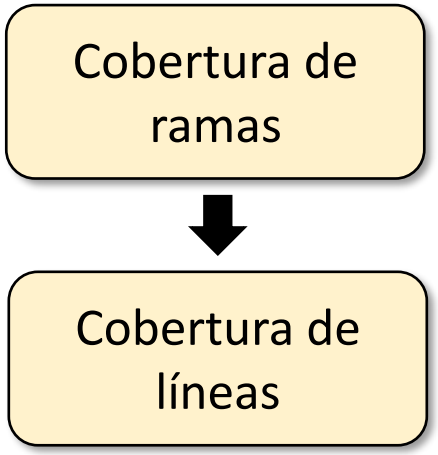
Todo conjunto de pruebas que cumple A automáticamente cumple B.

Es decir:

Si obtienes 100% en A, también obtienes 100% en B.

Ramas vs sentencias

- La cobertura de ramas es más fuerte que la de sentencias.
- En general, es más costoso alcanzar 100% cobertura de ramas.
- En el ejemplo anterior, podemos alcanzar 100% cobertura de sentencias, pero con el mismo test suite no conseguimos alcanzar un 100% de cobertura de ramas.



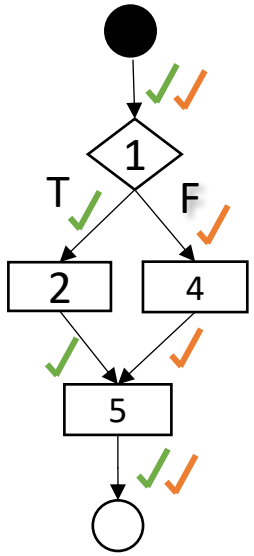
4. Pruebas de caja blanca – Métricas – Ramas

La cobertura en ramas funciona bien cuando no hay condiciones compuestas, pero…

```
    void func(float x, float y) {
1     if (x == 0 || y > 0) {
2       y = y / x;
    } else {
3       x = y + 2;
    }
4   System.out.println(x);
5   System.out.println(y);
    }
```

Caso de Prueba #1
x = 5
y = 5 ✓

Caso de Prueba #2
x = 5
y = -5 ✓



Cobertura de ramas = 100%
¡Pero hay un bug!

4. Pruebas de caja blanca – Métricas – Condiciones

- Mide qué porcentaje de condiciones booleanas han sido evaluadas.
- Cada condición debe evaluarse al menos una vez como **TRUE** y una vez como **FALSE**.
- NOTA: No mide la ejecución de sentencias (sólo condiciones).

```
public void aplicarDescuento (boolean esVIP, boolean fiel) {
  if (esVIP && fiel) {
      return true;
  }
}
```

esVIP	fiel	Resultado	¿Qué cubre?
True	False	False	esVIP = True fiel = False
False	True	False	esVIP = False fiel = True

4. Pruebas de caja blanca – Métricas – Ramas y condiciones

$$cobertura\ de\ ramas\ y\ condiciones = \frac{ramas\ ejecutadas + condiciones\ cubiertas}{número\ de\ ramas + número\ de\ condiciones} \cdot 100$$

Exige:

- Cada **rama** de cada decisión se ejecute al menos una vez (**TRUE** y **FALSE**).
- Cada **condición individual** se evalúe al menos una vez (**TRUE** y **FALSE**).

Garantiza:

- Que se recorren todas las ramas del flujo de control.
- Que cada condición simple ha sido ejercitada en ambos valores.

No garantiza:

- Que cada condición pueda afectar por sí sola al resultado de la decisión.
- Que se hayan probado todas las combinaciones posibles.

4. Pruebas de caja blanca – Métricas – Ramas y condiciones

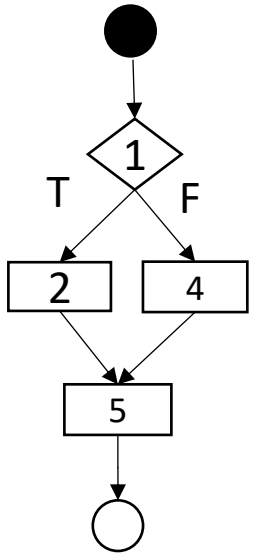
¿Cómo conseguir el 100% de la cobertura de ramas y condiciones?

- Falta un caso de prueba

```
   void func(float x, float y) {
1    if (x == 0 || y > 0) {
2      y = y / x;
     } else {
3      x = y + 2;
     }
4    System.out.println(x);
5    System.out.println(y);
   }
```

Caso de Prueba #1
x = 5
y = 5

Caso de Prueba #2
x = 5
y = -5



$$cobertura\ de\ ramas = \frac{2+3}{2+4} \cdot 100 = 83\%$$

4. Pruebas de caja blanca – Métricas – Ramas y condiciones

¿Cómo conseguir el 100% de la cobertura de ramas y condiciones?

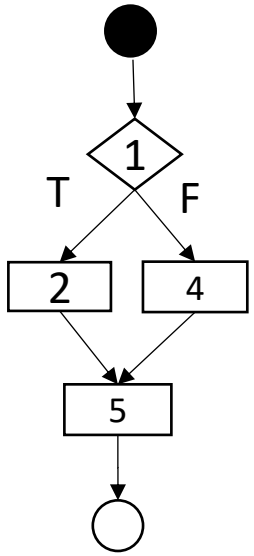
- Condición x == 0 => True

```
   void func(float x, float y) {
1    if (x == 0 || y > 0) {
2      y = y / x;
     } else {
3      x = y + 2;
     }
4    System.out.println(x);
5    System.out.println(y);
   }
```

Caso de Prueba #1
x = 5
y = 5

Caso de Prueba #2
x = 5
y = -5

Caso de Prueba #3
x = 0
y = <cualquiera>



4. Pruebas de caja blanca – Métricas – Ramas y condiciones

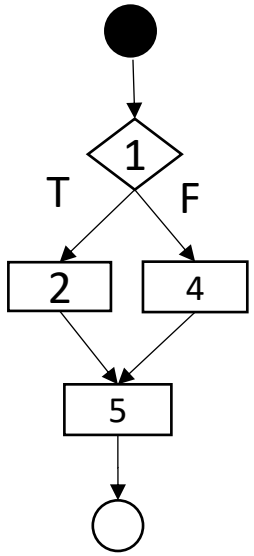
¿Cómo conseguir el 100% de la cobertura de ramas y condiciones?

- ¿Qué caso de prueba falta en este caso?

```
   void func(float x, float y) {
1    if (x == 0 || y > 0) {
2      y = y / x;
     } else {
3      x = y + 2;
     }
4    System.out.println(x);
5    System.out.println(y);
   }
```

Caso de Prueba #1
x = 0
y = -5

Caso de Prueba #2
x = 5
y = 5



4. Pruebas de caja blanca – Métricas – Condiciones múltiples

- Las pruebas de cobertura más potentes consisten en **comprobar todas las posibles combinaciones de condiciones** en cada decisión.
- Sin embargo, esta estrategia es impráctica:
  con 3 condiciones: 2^3 = 8 combinaciones.
  con 4 condiciones: 2^4 = 16 combinaciones.
  con 5 condiciones: 2^5 = 32 combinaciones.
  con 6 condiciones: 2^6 = 64 combinaciones.

4. Pruebas de caja blanca – Métricas – MCDC

Imagina que tenemos una condición compuesta:

```
if (A && B)
```

Sabemos que:

- Ramas exige que **`if`** sea **TRUE** y **FALSE**.
- Condiciones que **A** y **B** sean **TRUE** y **FALSE**.

Pero hay un problema:

Podrías cumplir ambos criterios sin demostrar que **A** realmente influye en el resultado, o que **B** influye.

Test	A	B	Salida
1	True	False	False
2	False	True	False

Aquí tenemos:

- **A** fue **TRUE** y **FALSE**.
- B fue **TRUE** y **FALSE**.
- La rama **FALSE** se ejecutó.

Pero:

- No probamos **TRUE** && **TRUE**.
- No hemos probado que cambiar solo **A** cambie el resultado.

* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Métricas – MCDC

Modified Condition Decision Coverage (MC/DC)

Cada condición debe demostrarse que puede afectar de forma independiente al resultado de la decisión.

Estrategia

1. Elegir aquellas condiciones que puedan, independientemente de las otras condiciones, afectar al resultado de la expresión.
2. Mantener los valores de las otras condiciones y comprobar si al cambiar el valor de la primera condición cambia el resultado completo.

Así, para cada condición debe existir un par de tests, donde solo cambia esa condición y el resultado global cambia.

MC/DC no quiere que pruebes todo, sino que demuestres que cada condición importa.

* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Métricas – MCDC

Código

```
if (A && B)
```

- Esto son 4 combinaciones con condiciones múltiples.
- MC/DC exige demostrar que cada condición puede cambiar el resultado por sí sola.

Test case	A	B	Salida
1	True	True	True
2	True	False	True
3	False	True	True
4	False	False	False

Vamos a ir seleccionando ejemplos poco a poco.

* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Métricas – MCDC

Código

```
if (A && B)
```

- Esto son 4 combinaciones con condiciones múltiples.
- MC/DC exige demostrar que cada condición puede cambiar el resultado por sí sola.

Test case	A	B	Salida
1	True	True	True
2	True	False	True
3	False	True	True
4	False	False	False

Seleccionamos nuestro primer caso de prueba. Vemos que el resultado de nuestra operación es **TRUE**.

* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Métricas – MCDC

Código

```
if (A && B)
```

- Esto son 4 combinaciones con condiciones múltiples.
- MC/DC exige demostrar que cada condición puede cambiar el resultado por sí sola.

Test case	A	B	Salida
1	True	True	True
2	True	False	False
3	False	True	True
4	False	False	False

Seleccionamos nuestro segundo caso de prueba.

- Este caso demuestra que modificar **B** afecta al resultado global.

* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Métricas – MCDC

Código

```
if (A && B)
```

- Esto son 4 combinaciones con condiciones múltiples.
- Con MC/DC, hemos necesitado 3 casos en lugar de 4.
- MC/DC requiere N + 1 casos.

Test case	A	B	Salida
1	True	True	True
2	True	False	False
3	False	True	False
4	False	False	False

Seleccionamos nuestro tercer caso de prueba.

- Este caso demuestra que modificar **A** también afecta al resultado global.

* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Métricas – MCDC

Criterio importante:

La FAA *(Federal Aviation Administration)* obliga a que todo el software que se ejecuta en aviones comerciales esté probado de acuerdo con el criterio MC/DC.

The untold story of SQLite

https://corecursive.com/066-sqlite-with-richard-hipp/

I'm going to write tests to bring SQLite up to the quality of 100% MCDC, and that took a year of 60 hour weeks. That was hard, hard work. I was putting in 12 hour days every single day.

4. Pruebas de caja blanca – Métricas – MCDC

A && B && C

Test case	A	B	C	Salida
1	True	True	True	True
2	True	True	False	False
3	True	False	True	False
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Casos de prueba seleccionados: {1, 5}, {1,3}, {1, 2} => 1, 5, 3, 2

* https://www.youtube.com/watch?v=DivaWCNohdw

4. Pruebas de caja blanca – Métricas – MCDC

```
A && (B || C)
```

Test case	A	B	C	Salida
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

A = {1, 5}, {2, 6}, {3, 7}
B = {2, 4}
C = {3, 4}

→ Selección: {2, 4, 3, 6}

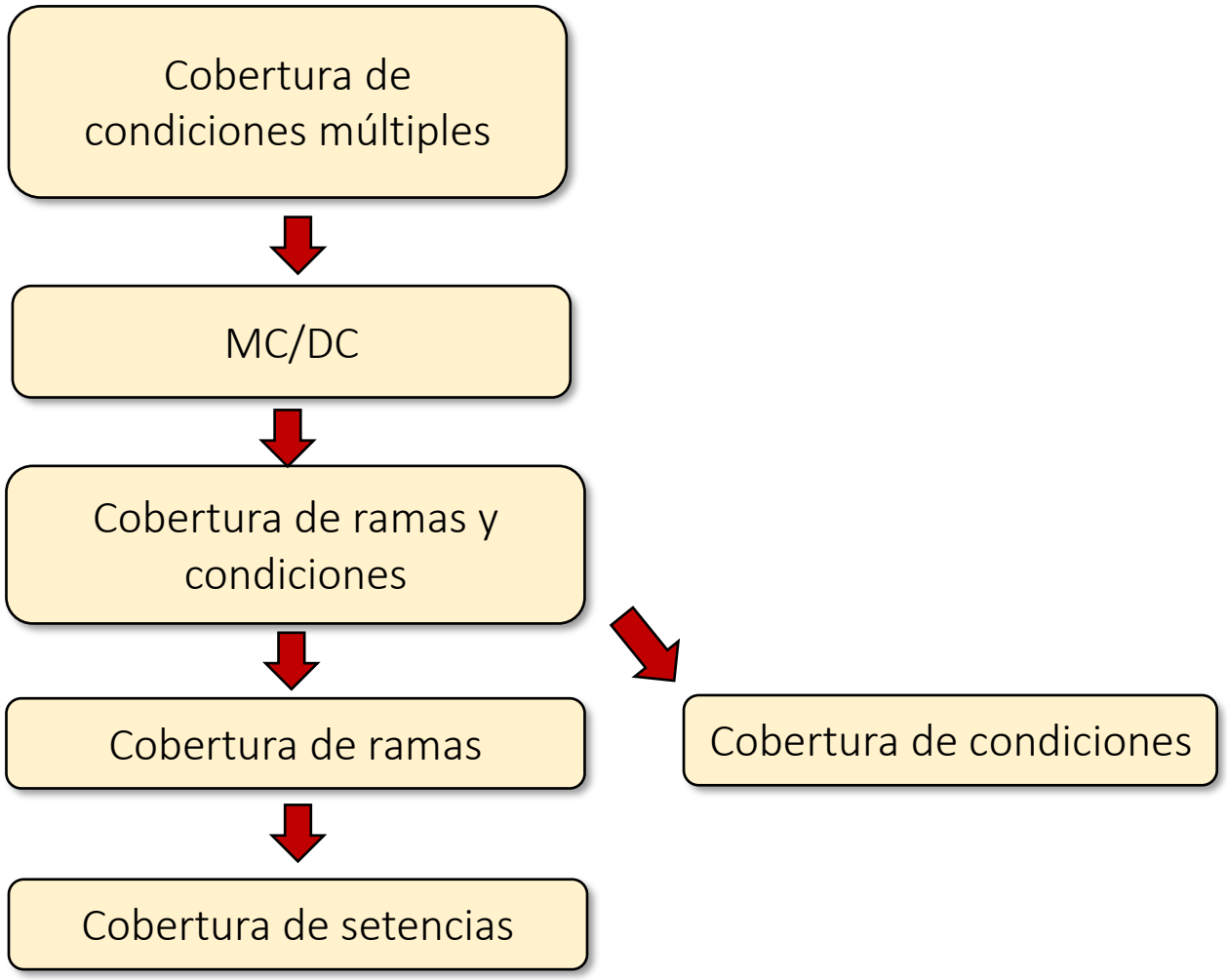
* https://www.youtube.com/watch?v=HzmnCVaICQ4

4. Pruebas de caja blanca – Criterios

Un criterio es más fuerte si implica cumplir los criterios inferiores.

De más a menos a fuerte:

- Cobertura de condiciones múltiples.
- MC/DC.
- Cobertura por ramas y condiciones.
- Cobertura por condiciones.
- Cobertura de ramas.
- Cobertura de sentencias.



4. Pruebas de caja blanca – Resumen

Métrica	Nivel de exigencia	¿Qué mide?
Sentence coverage (📏)	Básico	Ejecutar cada sentencia al menos una vez.
Condition coverage (🔦)	Medio	Que cada condición se evalúe como `true` y `false`.
Branch coverage (🌿)	Medio	Tomar cada rama (`true`/`false`) al menos una vez.
Branch + Condition (🌿 + 🔦)	Alto	Tomar cada rama tanto como `true` y `false`.
MC/DC (🎯)	Muy alto	Cada condición debe afectar sola al resultado.
Multiple Condition (🧠)	Extremadamente alto	Todas las combinaciones posibles de condiciones.

4. Pruebas de caja blanca – Bucles

No es possible probar todas las iteraciones de un bucle.

- Necesitamos un criterio de cobertura
- "*Loop boundary adequacy criterion*":
  - Hay una caso de prueba que itera 0 veces.
  - Hay un caso de prueba que itera una vez.
  - Hay un caso de prueba que itera más de una vez.
- En la práctica, tratar de combinarlo con otros criterios de cobertura de ramas y/o condiciones.
  - Ej:, ¿Qué pasa con un bucle que tiene un **`break`** dentro?

5. Buenas prácticas y herramientas.

5. Buenas prácticas – Recomendaciones

- Pruebas aisladas y que no comparten ningún estado, se pueden ejecutar en cualquier orden.
- Considera casos de fallo, no solo de éxito.
- Pon un único aserto por prueba unitaria, aunque no siempre es posible.
- Evita programar con código condicional (`if-then`)
- Sólo crea objetos de la clase sujeta a testing.
- Evitar usar números mágicos en las pruebas: usa valores con significado explícito.
- Nombra constantes cuando el valor no sea evidente (Ejemplo: `EdadMinima`).
- El test debe ser legible y autoexplicativo.
- Mide la cobertura con herramientas como JaCoCo, Sonar, Clover, ..

https://medium.com/vx-company/the-5-unit-testing-guidelines-f21d39c33e0b

5. Buenas prácticas – Testabilididad

Capacidad del software para ser probado

- Facilidad con la que una aplicación o módulo puede ser probado.
- Depende de la estructura del código, modularidad o desacoplamiento.

Factores que afectan:

- **Acoplamiento**: Código acoplado es difícil de probar.
- **Modularidad**: Clases y métodos bien definidos facilitan las pruebas.
- **Dependencias globales.**
  Variables estáticas o instancias creadas dentro de una clase dificultan los tests. También los *singleton*
- **Accesibilidad**: Métodos privados o código con muchas dependencias ocultas son difíciles de testear.
  `Anotación @VisibleForTesting`

5. Buenas prácticas – Inyección de dependencias

La inyección de dependencias facilita la testabilidad.

- Permite pasar las dependencias necesarias a una clase en lugar de que la clase las cree por sí misma.
- Se usa para desacoplar componentes y mejorar la testabilidad.

```
public class Cliente {
    private final Servicio servicio;
    public Cliente() {
        this.servicio = new Servicio();
        // this.servicio = Servicio.INSTANCE
    }
}
```

```
public class Cliente {
    private final Servicio servicio;
    public Cliente(Servicio servicio) {
        this.servicio = servicio;
    }
}
```

```
public class Cliente {
    @Inject
    private final Servicio servicio;
}
```

5. Buenas prácticas – Mocks

- Hasta ahora… los métodos y las clases estaban aislados unos de otros.
  - Configurar un objeto.
  - Determinar los parámetros.
  - Invocar y probar el resultado.

- ¿Qué sucede cuando los objetos tienen dependencias complejas?
  - ¿Crear todo el grafo de objetos?
  - Puede ser muy costoso

- ¿Qué sucede cuando los objetos dependen de servicios externos?
  - Por ejemplo, una base de datos.

5. Buenas prácticas – Mocks

Dummy object.

- Objetos que se pasan a la clase que se va a probar, pero que no se usan
- Ej., `new Cliente("prueba@um.es", new Direccion("c/prueba"))` n
- Para probar el método cliente.esVip() no nos hagan falta el correo ni la dirección

Fake object.

- Objetos que tienen una implementación "simulada"
- Usan una implementación más simple (ej., una base datos como H2)
- Ej.

```
public class DatabaseFakeObject implements Database {
  private List<Cliente> clientes;
  @Override
  public List<Cliente> getClientes() {
     return clientes
  }
}
```

5. Buenas prácticas – Mocks

Stub.

- Similares a los Fake Objects pero en su lugar devuelven valores fijos
- Ej.,

```
public class DatabaseMock implements Database {
  @Override
  public List<Cliente> getClientes() {
     return List.of(new Cliente("c1@um.es", "c2@um.es"));
  }
}
```

- Los frameworks de *mocking* suelen permitir configurar los stubs para que no sea necesario crear las implementaciones (se configura un objeto con un comportamiento fijo).

5. Buenas prácticas – Mocks

Mock.

- Similares a los Stub, pero además guardan las interacciones que se han realizado sobre ellos y se pueden consultar.
- Ej.,

```
public class DatabaseStub implements Database {
  private numLlamadasGetClientes = 0;

  @Override
  public List<Cliente> getClientes() {
     numLlamadasGetClientes++;
     return List.of(new Cliente("c1@um.es", "c2@um.es"));
  }
  int getNumLlamadasGetClientes() { return numLlamadasGetClientes; }
}
```

- Los frameworks de *mocking* son capaces de guardar las interacciones

5. Buenas prácticas – Herramientas – Mockito

Es un framework de mocking para Java

Simular el comportamiento de dependencias sin necesidad de crear implementaciones reales.

- Creación de mocks:
  - Se puede hacer manualmente con `Mockito.mock(Clase.class)`.
  - También se pueden usar anotaciones como `@Mock`.
- Definir comportamientos:
  - Se usa `when(mock.metodo()).thenReturn(valor)` para establecer respuestas predefinidas.
  - También se puede lanzar excepciones con `thenThrow()`.
- Verificación de interacciones:
  - Con `verify(mock).metodo()` se puede comprobar si un método fue llamado.
  - Se pueden verificar cuántas veces se llamó (`times(n)`) o si nunca se usó (`never()`).

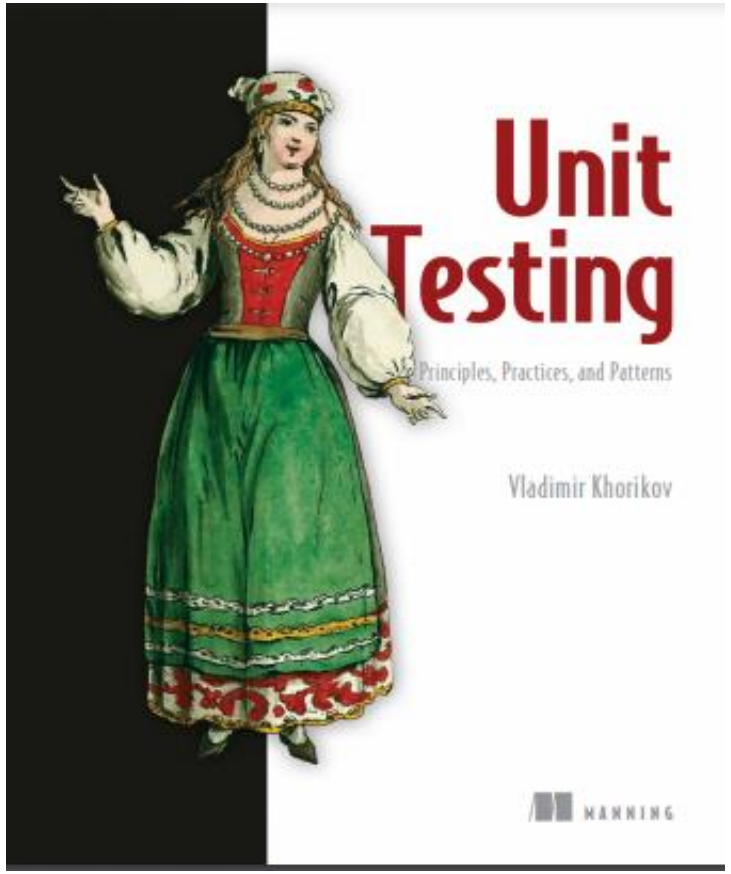
5. Buenas prácticas – Herramientas – Mockito

```java
public class ClienteTest {

    @Test
    void testProcesar() {
        // 1. Crear un mock para la clase Servicio
        Servicio servicioMock = Mockito.mock(Servicio.class);

        // 2. Definir el comportamiento del mock
        when(servicioMock.obtenerDato()).thenReturn("Dato simulado");

        // 3. Crear instancia de la clase que se va a probar
        //    y que tiene una dependencia con Servicio
        Cliente cliente = new Cliente(servicioMock);

        // 4. Ejecutar método y verificar resultado
        String resultado = cliente.procesar();
        assertEquals("Procesado: Dato simulado", resultado);

        // 5. Verificar que el método se llamó una vez
        verify(servicioMock, times(1)).obtenerDato();
    }
}
```

5. Buenas prácticas – Herramientas – jUnit

JUnit es un framework para pruebas unitarias en Java creado por Erich Gamma y Kent Beck.

- Software *Open Source* distribuido bajo la licencia *Common Public* 1.0.
- Puede descargarse gratuitamente de http://www.junit.org
- Facilita la creación y ejecución de pruebas unitarias, de integración y regresión.



5. Buenas prácticas – Herramientas – jUnit

```
import static org.junit.Assert.*;
import org.junit.Test;

public class VentaTest {

  @Test
  public void testGetTotal() {
    //Crear objetos sobre los que ejecutar test
    Dinero precio = new Dinero(2.5);
    ItemID id = new ItemId(1);
    Producto p = new Producto (id, precio, “producto 1”);
    Venta venta = new Venta();
    venta.addLineaVenta(p,1);
    venta.addLineaVenta(p,2);

    //Establecer resultado
    Dinero total = new Dinero(7.5);

    //Ejecutar asertos del test
    assertEquals(venta.getTotal(), total);
  }
}
```

5. Buenas prácticas – Herramientas – jUnit

`assertEquals(Object expected, Object actual)`	Asserts that two objects are equal.
`assertEquals (double expected, double actual, double delta)`	Asserts that two doubles are equal to within a positive delta.
`assertNotEquals(Object unexpected, Object actual)`	Asserts that two object are not equal
`assertFalse(boolean condition)`	Asserts that a condition is false.
`assertTrue(boolean condition)`	Asserts that a condition is true.
`assertNotNull(Object object)`	Asserts that an object isn't null.
`assertNull(Object object)`	Asserts that an object is null.
`assertSame(Object expected, Object actual)`	Asserts that two objects refer to the same object.
`fail(String message)`	Fails a test with the given message.

5. Buenas prácticas – Herramientas – jUnit

`@Test`	Indicar qué un método es un caso de prueba
`@DisplayName`	Nombre que se mostrará para la clase o método de prueba
`@BeforeEach`	El método se ejecuta *antes de cada* caso de prueba (método con @Test)
`@AfterEach`	El método se ejecuta *después de cada* caso de prueba (método con @Test)
`@BeforeAll`	El método se ejecuta antes de que empiecen los casos de prueba de la clase
`@AfterAll`	El método se ejecuta cuando han terminado los casos de prueba de la clase
`@Disabled`	Deshabilitar un caso de prueba

* Listado completo en https://junit.org/junit5/docs/current/user-guide/#writing-tests-annotations

5. Buenas prácticas – Herramientas – jUnit

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvFileSource;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

  @ParameterizedTest
  @CsvFileSource(
    resources = "/doblar-casos.csv",
    numLinesToSkip = 1
  )
  void testDoblar(int entrada, int esperado) {
    assertEquals(esperado, Calculadora.doblar(entrada));
  }
}
```

5. Buenas prácticas – Herramientas – SpringBoot

En Spring Boot hay distintos niveles:

- Tests unitarios.
- Tests de integración.
- Tests de persistencia.
- Tests de integración con contexto Spring completo.

5. Buenas prácticas – Herramientas – SpringBoot

Tests unitarios.

El dominio no necesita Spring para probarse.

- No se levanta contexto.
- No se usan anotaciones especiales.

```
class PedidoServiceTest {
  PedidoService service = new PedidoService();
  @Test
  void aplicaDescuentoVIP() {
    assertEquals(10, service.calcularDescuento(true));
  }
}
```

5. Buenas prácticas – Herramientas – SpringBoot

Tests de integración.

- Hay pruebas para evaluar los endpoints de manera independiente.
- Sin levantar la aplicación.
- Sin arrancar el servidor.
- Sin una base de datos.

5. Buenas prácticas – Herramientas – SpringBoot

Tests de integración.

- Por ejemplo:
  Controlador → Servicio → Repositorio
- Vamos a probar el controlador **PedidoController**.

```
@RestController
@RequestMapping("/pedidos")
class PedidoController {
  private final PedidoService service;
  public PedidoController(PedidoService service) {
    this.service = service;
  }

  @GetMapping("/{id}")
  public PedidoDTO obtener(@PathVariable String id) {
    return service.obtener(id);
  }
}
```

```
@WebMvcTest(PedidoController.class)
class PedidoControllerTest {

  @Autowired
  private MockMvc mockMvc;

  @MockBean
  private PedidoService service;

  @Test
  void devuelvePedido() throws Exception {

    when(service.obtener("1"))
      .thenReturn(new PedidoDTO("1"));

    mockMvc.perform(get("/pedidos/1"))
      .andExpect(status().isOk())
      .andExpect(jsonPath("$.id").value("1"))
    ;
  }
}
```

Bibliografía

- **Software Engineering – Basic Principles and Best Practices**. Ravi Sethi. Cambridge University Press. 2023.
  - Capítulo9
- **Effective Software Testing**. Mauricio Aniche.
  - Capítulos 2, 3 y 6